

根据课程资料, vLLM 和 SGLang 在处理约束解码(如强制输出特定格式的 JSON)时, 核心差异在于底层实现机制以及准确率的保证。

以下是它们的主要不同点:

1. 底层实现机制

- **vLLM(传统方式/现状):**
 - 主要依赖 **“Prompt 工程 + 后校验”** 的方式来生成结构化数据。
 - 虽然 vLLM 现在也开始集成像 outlines 这样的库来做约束, 但课程录音中提到, 如果它底层不是完全基于有限状态机(FSM)严格推导出来的, 往往会采用一些后置校验或模拟的方法。
- **SGLang(原生架构优化):**
 - 采用 原生约束解码, 底层直接编译并运行压缩有限状态机(**Compressed FSM**)。
 - 在解码前向传递时, 系统将正则表达式转换为状态机, 直接在物理层面把不符合规则(如非法字符、缺失标点)的 Token 生成概率强制设置为零。

2. 输出准确率与生产稳定性

- **vLLM (Prompt模式):**
 - 格式正确率大约在 **90% - 95%** 左右。
 - 依赖模型自身的概率生成, 总有极小概率会意外丢掉一个引号或逗号, 导致下游系统 JSON 解析崩溃。
 - 在生产环境中, 通常必须额外编写 try-except 错误捕获和复杂的重试逻辑。
- **SGLang (FSM模式):**
 - 格式正确率保证 **100%**。
 - 一次生成即是合法结构, 彻底免去了错误捕获和繁琐的重试环节, 生产环境极度稳定(“睡觉安心”)。

3. 解码速度与性能

- **传统约束系统:**在处理约束时通常只能一次解码一个 Token, 遇到需要输出长序列的固定字符串时(比如 JSON 的 key "summary":), 效率非常低下。
- **SGLang 的 压缩有限状态机 (Compressed FSM):**
 - 它分析并将 FSM 中相邻的单次转换边压缩成单个边。

- 这种设计允许系统识别出哪些固定字符可以一起输出，从而在一个前向传递中一次性解码多个 **Token**，大幅加速了复杂约束任务的解码速度。

总结来说，对于简单的通用对话，vLLM 就足够了；但如果你的业务场景是 Agent 调用，且严格要求大模型必须输出标准、无错的 JSON 给下游系统消费，SGLang 的压缩状态机约束解码会比 vLLM 稳得多，速度也更快。